Team Presentation

Team 5

Qingdao University of Technology

October 4, 2022

Problem 1

Design two nonlinear algebraic equations (degree at least 5 , with no less than 3 zeros) and find all their zeros (error less than 0.001),show your result and Matlab or Python code.

Problem 1 Solution

First, draw the graph of the higher degree equation of one variable then find the approximate interval between zero, Finally, the binary method is used to determine the value of zero.
Following are our target functions:

$$y = x^6 + x^4 - 3x^5 - 2x^3 + 4x^2 \tag{1}$$

$$y = x^7 - 2x^6 - 4x^5 + 3x^3 + 3x^2 \tag{2}$$

Problem 1 Matlab Code

```
function r=bisect(a,b,accurate)
p=(a+b)/2;
error=b-a;n=0;
while (error>accurate)
    n=n+1;
    if fun(p)==0
      break;
          else if fun(a)*fun(p)<0
            b=p;
        else
            a=p;
              end
        p=(a+b)/2;
        error=(b-a)/2^2;
    end
          r=p;
end end
```

Problem 1 Solution

By using bisection algorithm, we got the approximation of the function's root:

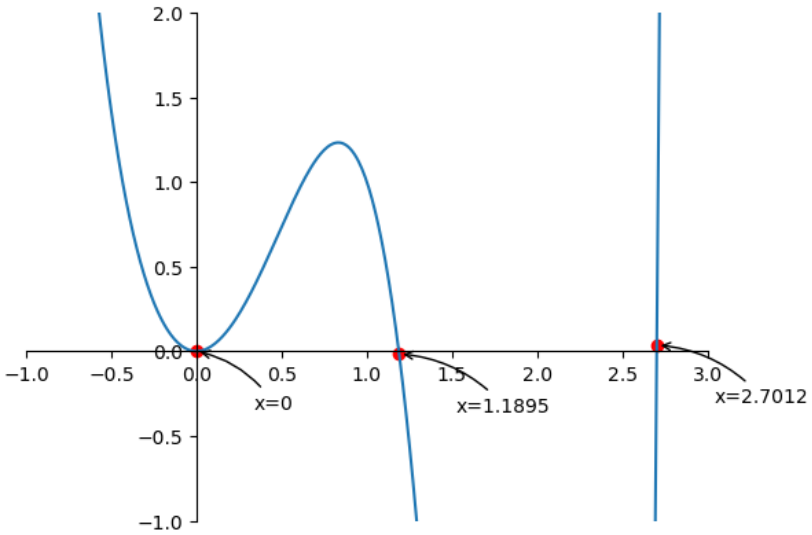

Figure: $y = x^6 + x^4 - 3x^5 - 2x^3 + 4x^2$

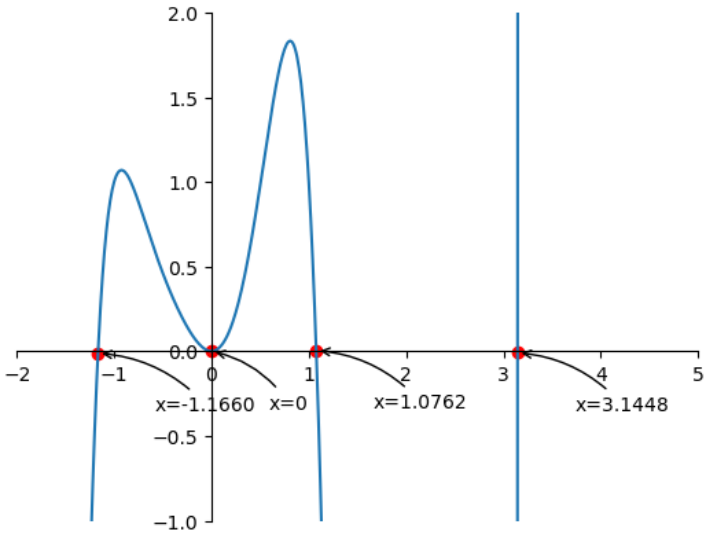

Figure: $y = x^7 - 2x^6 - 4x^5 + 3x^3 + 3x^2$

Problem 2(1)

Find the numerical solution of the following nonlinear equations using iteration method and Newton method respectively and show your analysis of error by using norms of vector and matrix and demonstrate your computer code

$$\begin{cases} x_1^2 - 10x_1 + x_2^2 + 8 = 0 \\ x_1 x_2^2 + x_1 - 12x_2 + 7 = 0 \end{cases} \tag{3}$$

Problem 2(1) Solution

By using $\Delta X^{(k)} = X^{(k+1)} - X^{(k)}$,
we had

$$X^{(1)} = \Delta X^{(0)} + X^{(0)} = \begin{pmatrix} 0.470588 \\ 0 \\ -0.352941 \end{pmatrix} \tag{4}$$

The rest can be done in the same manner. Following are each iteration result.

	1	2	3	4
x_1	0.470588	0.423831	0.424716	0.4247
x_2	0	-0.042279	-0.041371	-0.0413
x_3	-0.352941	-0.288278	-0.286962	-0.2870

Problem 2(1) Fixed-Point Python Code

```
x = np.zeros([max_iteration, 2], dtype='float64')
err = np.zeros(max_iteration - 1, dtype='float64')
distance = 1;idx = 1;x[0, 0] = 1;x[0, 1] = 1
while(distance > acc):
    if(idx < max_iteration):
        x[idx, 0] = ((x[idx - 1, 0])**2 + (x[idx - 1, 1])**2 + 8) / 10
        x[idx, 1] = (x[idx - 1, 0] + (x[idx - 1, 0] * x[idx - 1, 1])**2 + 7) / 12
        distance = (x[idx - 1, 0] - x[idx, 0])**2 + (x[idx - 1, 1] - x[idx, 1])**2
        err[idx - 1] = distance
        idx = idx + 1
    else:
        break
```

Problem 2(1)

In a very extreme condition like in a tolerance of 1×10^{-20}, the fixed-point iteration took about 20 times to reach the target.

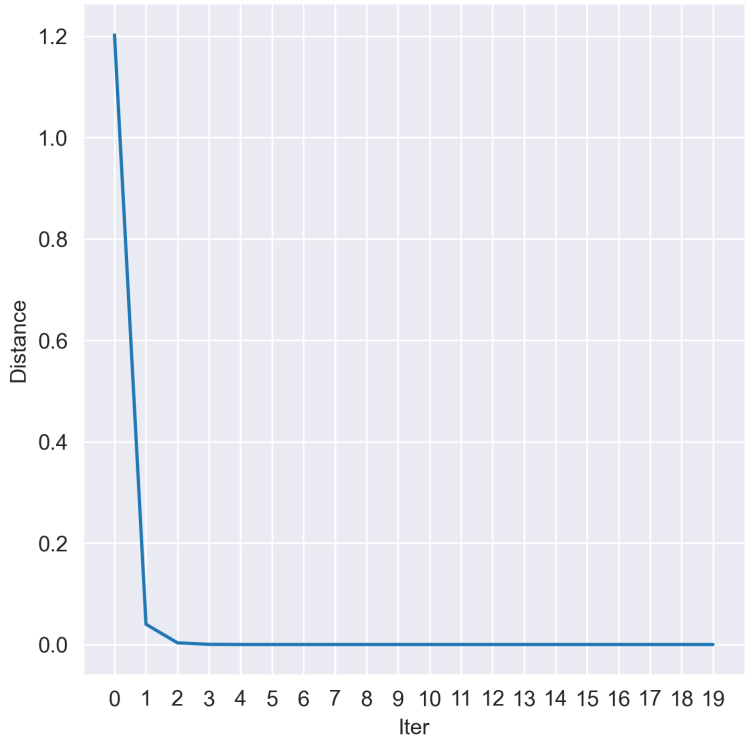

Figure: Fixed-Point Iteration Result

Problem 2(1) Newton-Raphson Matlab Code

```
syms x1 x2;
f1(x1,x2) = x1^2 - 10*x1 + x2^2 + 8;
f2(x1,x2) = x1*x2^2 + x1 - 12*x2 + 7;
F(x1,x2) = [f1;f2];x_solve(:,1) = [1;1];
J(x1,x2) = jacobian(F,[x1 x2]);
dx_Nr =[0;0];
for i = 1:N
    dx_Nr(:,i) = J(x_solve(1,i),x_solve(2,i))\(-F(x_solve(1,i),x_solve(2,i)));
    if(sqrt(sum(dx_Nr(:,i).^2)) < epsilon)
        x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
        break;
    else
        x_solve(:,i+1) = x_solve(:,i) + dx_Nr(:,i);
    end end
```

Problem 1 Solution

Using programing tools, we got the root of this given nonlinear system. In the tolerance of $1 \times 10^{-}10$, the two method showed a very different performance. Newton-Raphson iteration converge in incredible speed which only took 4 times to get a trustable root.

$$\begin{cases} x_1 = 0.936680033263737 \\ x_2 = 0.699593344681334 \end{cases} \tag{5}$$

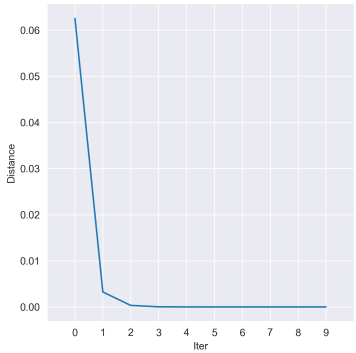

Figure: Fixed-Point Iteration

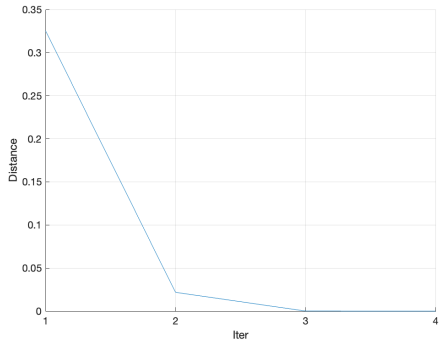

Figure: Newton-Raphson Iteration

Problem 1 Solution

Using programing tools, we got the root of this given nonlinear system. In the tolerance of $1 \times 10^{-}10$, the two method showed a very different performance. Newton-Raphson iteration converge in incredible speed which only took 4 times to get a trustable root.

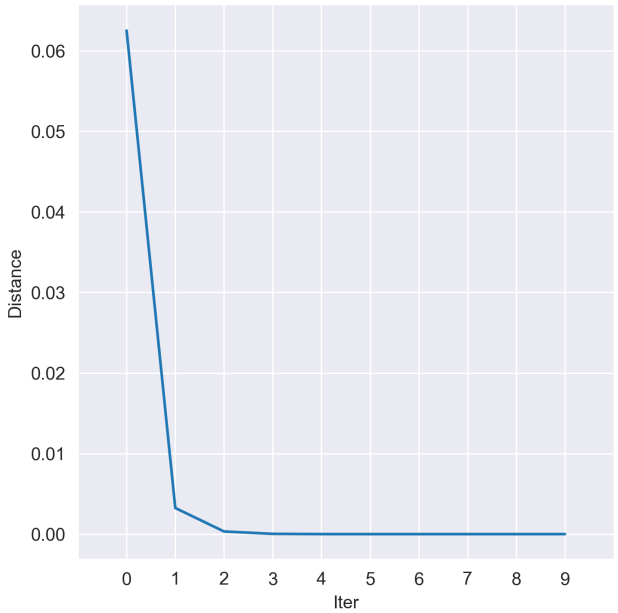

Figure: Fixed-Point Iteration

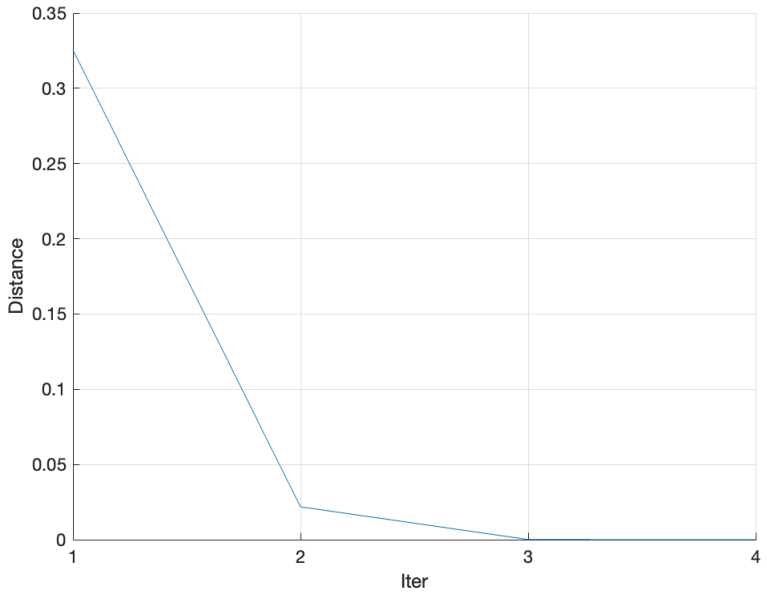

Figure: Newton-Raphson Iteration

Problem 2(2)

Find the numerical solution of the following nonlinear equations using iteration method and Newton method respectively and show your analysis of error by using norms of vector and matrix and demonstrate your computer code

$$\begin{cases} 5x_1 - x_2^2 + x_1x_3 = 2 \\ x_1x_3 + 4x_2 - x_3 = 0 \\ x_1^2 + x_2 - 3x_3 = 1 \end{cases} \tag{6}$$

Problem 2(2) Solution

We let $\mathrm{X} = \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix}$ and $\mathrm{F(X)} = \begin{pmatrix} 5x_1 - x_2^2 + x_1x_3 - 2 \\ x_1x_3 + 4x_2 - x_3 \\ x_1^2 + x_2 - 3x_3 - 1 \end{pmatrix}$. So the Jacobian matrix of $F(X)$ eqauls to

$$\begin{pmatrix} 5 + x_3 & -2x_2 & x_1 \\ x_3 & 4 & -1 \\ 2x_1 & 1 & -3 \end{pmatrix} \tag{7}$$

Problem 2(2) Solution

At first we guess $X^{(0)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}$,

so we got $\mathrm{F}(X^{(0)}) = \begin{pmatrix} 3 \\ 0 \\ 0 \end{pmatrix}$, $\mathrm{J}\left(X^{(0)}\right) = \begin{pmatrix} 0 & 0 & 1 \\ 0 & 4 & -1 \\ 2 & 1 & -3 \end{pmatrix}$.

By using $\mathrm{J}\left(X^{(n)}\right)\Delta X^{(n)} = -\mathrm{F}\left(X^{(n)}\right)$,

we have $\begin{pmatrix} 0 & 0 & 1 \\ 0 & 4 & -1 \\ 2 & 1 & -3 \end{pmatrix}\Delta X^{(0)} = \begin{pmatrix} 3 \\ 0 \\ 0 \end{pmatrix}$.

Thus, the solution is $\Delta X^{(0)} = \begin{pmatrix} -0.439412 \\ 0 \\ -0.352941 \end{pmatrix}$.

Problem 2(2) Solution

Next,by using $\Delta X^{(k)} = X^{(k+1)} - X^{(k)}$,
we had

$$X^{(1)} = \Delta X^{(0)} + X^{(0)} = \begin{pmatrix} 0.470588 \\ 0 \\ -0.352941 \end{pmatrix} \tag{8}$$

The rest can be done in the same manner. Following are each iteration result.

	1	2	3	4
x_1	0.470588	0.423831	0.424716	0.4247
x_2	0	-0.042279	-0.041371	-0.0413
x_3	-0.352941	-0.288278	-0.286962	-0.2870

Problem 2(2) Solution

By using norms of vector and matrix, here exists

$$\left\|X^{(n)} - X^{(0)}\right\| \leq \left\| \begin{array}{ccc} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{array} \right\| \wedge \mathrm{n} \left\|X^{(1)} - X^{(0)}\right\|, \tag{9}$$

and we got

$$\left\|X^{(4)} - X^{(0)}\right\| \leq \left\| \begin{array}{ccc} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \frac{\partial f_1}{\partial x_3} \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \frac{\partial f_2}{\partial x_3} \\ \frac{\partial f_3}{\partial x_1} & \frac{\partial f_3}{\partial x_2} & \frac{\partial f_3}{\partial x_3} \end{array} \right\| \wedge 4 \left\|X^{(1)} - X^{(0)}\right\| \leq 10^{-5} \tag{10}$$

Problem 2(2) Matlab Code

```
syms x1 x2 x3
f1(x1,x2,x3) = 5*x1-x2^2+x1*x3-2;
f2(x1,x2,x3) = x1*x3+4*x2-x3;
f3(x1,x2,x3) = x1^2+x2-3*x3-1;
f(x1,x2,x3) = [f1 f2 f3];
jac(x1,x2,x3) = jacobian([f1 f2 f3],[x1 x2 x3]);
p = [1 0 0];
F0 = [0 0 0];
for i=1:20
    dq = eval(F0-formula(f(p(1),p(2),p(3))));
    if norm(dq) < 1e-5
        fprintf('求解成功\t迭代次数:%d\n',i);
        p
        break;
    end
    dq = eval(formula(jac(p(1),p(2),p(3)))\dq');
    p = p+dq';
    fprintf('\n迭代次数:%d',i);
    vpa(p,10)
end
```

Problem 3(1)

Generate matrix $A(50 \times 50)$ and vector $b(50 \times 1)$ by using random number and solve the linear system $AX = b$ by using Gaussian partial pivoting elimination algorithm.

Problem 3(2)

Randomly generate 3 diagonal matrices (30 * 30), and use Jacobi iteration and Gauss Seidel iteration to solve them respectively. The error of the solution in terms of norm is required to be less than 1/1000.

```
%随机生成三对角矩阵
n=30;
A=diag(round(rand(1,n)*9)+1,0)+diag(round(rand(1,n-1)*9)+1,1)+diag(round(rand(1,n-1)*9)+1,-1);
b=round(rand(n,1)*9)+1;
x0=zeros(n,1);
%误差
eps=1e-3;
%最大迭代次数
Nmax=500;
```

Problem 3(2) Jacobi Matlab Code

```
%Jacobi
x = zeros(1,n);
y = zeros(1,n);
count = 1;
while(1)
     for i = 1:n
        for j = 1:n
            if i ~= j
                y(i) = y(i) + A(i,j)*x(j);
            end
        end
        y(i)=( b(i) - y(i))/A(i,i);
    end
    if max(abs(x-y)) < eps
        fprintf('迭代结束，次数%d，结果：\n',count);
        disp(y);
        break;
    end
    if count == Nmax
        fprintf('超过最大迭代次数，结果：\n');
        disp(y);
        break;
    end
    count = count + 1;
    x = y;
    y(1: n) = 0;
end
```

Problem 3(2) Gauss-Deidel Matlab Code

```
%Gauss-Seidel
x = zeros(1,n);
y = zeros(1,n);
count = 1;
x1 = zeros(1,n);
x2 = zeros(1,n);
while(1)
    x1(1:n) = 0;
    x2(1:n) = 0;
     for i = 1:n
        for j = 1: i-1
            x1(i) =x1(i) + A(i,j) * y(j);
        end
        for k = i + 1 : n
           x2(i) = x2(i) + A(i,k) * x(k);
        end
        y(i)=(b(i) - x1(i) - x2(i))/A(i,i);
    end
    if max(abs(x-y)) < eps
        fprintf('迭代结束，次数%d，结果：\n',count);
        disp(y);
        break;
    end
    if count == Nmax
        fprintf('超过最大迭代次数，迭代结束，结果：\n');
        disp(y);
        break;
    end
    count = count + 1;
    x = y;
end
```

Problem 4

Use appropriate polynomial interpolation to approximate the following ellipse.

$$\frac{x^2}{5^2} + \frac{y^2}{5^2} = 1 \tag{11}$$

$$x^2 + y^2 = 5^2 \tag{12}$$

Problem 5

我国古代数学著作《九章算数》记载的线性方程组如下：

今有上禾三秉,中禾二秉,下禾一秉,实三十九斗；上禾二秉,中禾三秉,下禾一秉,实三十四斗；

上禾一秉,中禾二秉,下禾三秉,实二十六斗；问上、中、下禾实一秉各几何?

查阅九章算术记载的此问题的解法是什么？指出相当于今天的什么方法。

Problem 5

我国古代数学著作《九章算数》记载的线性方程组如下：

今有上禾三秉,中禾二秉,下禾一秉,实三十九斗；上禾二秉,中禾三秉,下禾一秉,实三十四斗；

上禾一秉,中禾二秉,下禾三秉,实二十六斗；问上、中、下禾实一秉各几何?

查阅九章算术记载的此问题的解法是什么？指出相当于今天的什么方法。